

TP - Doctolib & Les Echos

Piloter un navigateur reel : banniere cookies, WebDriverWait, scroll, headless. Deux cibles, deux niveaux de difficulte.

5 heures de developpement

Objectifs pedagogiques

A l'issue de ce TP vous serez capable de :

- Identifier quand Selenium est indispensable face a requests+BS4
- Gerer une banniere de consentement cookies (3 strategies)
- Utiliser WebDriverWait + ExpectedConditions plutot que time.sleep()
- Faire defiler une page (scroll) pour charger du contenu paresseux
- Activer le mode headless et mesurer le gain de performance
- Capturer un screenshot automatique en cas d'echec

LIVRABLES ATTENDUS

doctolib_scraper.py : export doctolib.json (>= 5 medecins)

lesechos_scraper.py : export lesechos.json (titres une)

screenshots/ : au moins 1 capture en cas d'echec detecte

README.md : comparaison temps headless vs normal, choix technique

Rappel architecture Selenium

Selenium 4.10+ integre Selenium Manager : plus besoin de telecharger chromedriver manuellement. La chaine de commande : script Python -> WebDriver -> ChromeDriver -> Chrome.

```
# Installation
pip install selenium

# Test rapide
from selenium import webdriver
driver = webdriver.Chrome()          # Selenium Manager gere chromedriver
driver.get("https://example.com")
print(driver.title)
driver.quit()
```

Rappel : 6 strategies de localisation

By.ID : id='search-input'

By.CSS_SELECTOR : 'div.results a.card' (recommande)

By.XPATH : '//button[contains(text(),"Accepter")]'

By.NAME : name='q'

By.LINK_TEXT : 'Voir plus de resultats'

By.TAG_NAME : 'h1' (peu selectif, usage rare)

TD 2.1 - Doctolib : creneaux disponibles

2 h 30

Cible : www.doctolib.fr - recherchez une specialite medicale dans une ville de votre choix (ex: cardiologue a Lyon).

Extrayez les fiches medecins affichees en premiere page de resultats.

Etape 1 - Lancer et naviguer

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.keys import Keys
import time, json, os

options = webdriver.ChromeOptions()
# options.add_argument("--headless=new") # commenter pour headless
options.add_argument("--disable-blink-features=AutomationControlled")
options.add_experimental_option("excludeSwitches", ["enable-automation"])

driver = webdriver.Chrome(options=options)
wait = WebDriverWait(driver, 15)

URL = "https://www.doctolib.fr/cardiologue/lyon"
driver.get(URL)
```

Etape 2 - Gerer la banniere cookies

Doctolib affiche une banniere de consentement. Trois strategies, par ordre de preference :

```
# Strategie 1 (RECOMMANDEE) : cliquer sur le bouton Accepter
try:
    btn = wait.until(EC.element_to_be_clickable(
        (By.XPATH, '//*[@button[contains(text(),"Accepter") or contains(text(),"Tout accepter")]]')
    ))
    btn.click()
    print("Cookies acceptes")
except:
    print("Pas de banniere detectee")

# Strategie 2 : injecter le cookie directement (plus robuste)
# driver.add_cookie({"name": "cookie_consent", "value": "true", "domain": ".doctolib.fr"})

# Strategie 3 : profil Chrome persistant
# options.add_argument("--user-data-dir=/tmp/chrome_profile")
```

Etape 3 - Attendre les resultats

La page charge les resultats en JavaScript. Ne jamais utiliser `time.sleep()` fixe. Utilisez `WebDriverWait` pour attendre l'apparition d'un element fiable.

```
# Attendre que les cartes medecins soient visibles
try:
    wait.until(EC.presence_of_element_located(
        (By.CSS_SELECTOR, "div[data-test='search-result-card']")
    ))
    print("Resultats charges")
except Exception as e:
    # Capture screenshot pour debug
    os.makedirs("screenshots", exist_ok=True)
    driver.save_screenshot("screenshots/doctolib_erreur.png")
    raise RuntimeError(f"Resultats non charges : {e}")
```

Etape 4 - Scroll pour charger plus de resultats

```
# Defiler pour charger les resultats hors viewport
def scroll_to_bottom(driver, pauses=3):
    last_h = driver.execute_script("return document.body.scrollHeight")
    for _ in range(pauses):
        driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")
        time.sleep(1.5)
        new_h = driver.execute_script("return document.body.scrollHeight")
        if new_h == last_h:
            break
        last_h = new_h

scroll_to_bottom(driver)
```

Etape 5 - Extraire les 5 champs

Pour chaque carte medecin, extrayez les champs suivants. Adaptez les selecteurs CSS si necessaire (inspecter avec DevTools).

5 champs a extraire

1. nom_specialite : nom du medecin + specialite
2. adresse : adresse du cabinet
3. type_consultation: 'Cabinet' ou 'Video' ou les deux
4. prochains_creneaux : liste des 3 premiers creneaux disponibles
5. url_fiche : URL complete de la fiche praticien

```
def extraire_medecins(driver) -> list[dict]:
    cartes = driver.find_elements(By.CSS_SELECTOR, "div[data-test='search-result-card']")
    resultats = []
    for carte in cartes[:10]: # limiter a 10
        try:
            nom = carte.find_element(By.CSS_SELECTOR, "h2, h3, [class*='name']").text.strip()
            adr = carte.find_element(By.CSS_SELECTOR, "[class*='address']").text.strip()
            url_el = carte.find_element(By.CSS_SELECTOR, "a[href*='/praticien/']")
            url = url_el.get_attribute("href")
            creneaux = [
                el.text.strip()
                for el in carte.find_elements(By.CSS_SELECTOR, "[class*='slot']")[:3]
            ]
            types = [
                el.text.strip()
                for el in carte.find_elements(By.CSS_SELECTOR, "[class*='consultation-mode']")
            ]
            resultats.append({
                "nom_specialite": nom, "adresse": adr,
                "type_consultation": types or ["n/a"],
                "prochains_creneaux": creneaux,
                "url_fiche": url,
            })
        except Exception as e:
            print(f"Carte ignoree : {e}")
    return resultats

medecins = extraire_medecins(driver)
driver.quit()

with open("doctolib.json", "w", encoding="utf-8") as f:
    json.dump(medecins, f, indent=2, ensure_ascii=False)
print(f"{len(medecins)} medecins exportes dans doctolib.json")
```

TD 2.2 - Les Echos : titres a la une

2 h 30

Cible : www.lesechos.fr - extrayez les articles de la page d'accueil.

Etape 1 - Tester d'abord avec requests

Avant de lancer Selenium, testez si requests suffit. Si le contenu est present dans le HTML retourne par requests, Selenium est inutile.

```
import requests
from bs4 import BeautifulSoup

r = requests.get("https://www.lesechos.fr",
                 headers={"User-Agent": "IPSSI-scrapers (+contact@ipssi.fr)"},
                 timeout=10)
soup = BeautifulSoup(r.text, "lxml")
titres = soup.select("h2, h3")
print(f"{len(titres)} balises de titre trouvees")
# Si 0 => page chargee en JS => Selenium necessaire
```

Etape 2 - Selenium + headless

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import json, time

def make_driver(headless: bool = False):
    opts = webdriver.ChromeOptions()
    if headless:
        opts.add_argument("--headless=new")
        opts.add_argument("--no-sandbox")
        opts.add_argument("--disable-dev-shm-usage")
    opts.add_argument("--disable-blink-features=AutomationControlled")
    return webdriver.Chrome(options=opts)

# Mesurer le temps
import time
t0 = time.time()
driver = make_driver(headless=False)
driver.get("https://www.lesechos.fr")
WebDriverWait(driver, 15).until(
    EC.presence_of_element_located((By.CSS_SELECTOR, "article, [class*='article']"))
)
t_normal = time.time() - t0
print(f"Mode normal : {t_normal:.1f}s")
```

Etape 3 - Extraire 5 champs

5 champs a extraire

1. titre : texte du titre de l'article
2. rubrique : section / categorie (ex: Economie, Finance...)
3. chapeau : premier paragraphe ou description
4. heure_publi : heure de publication affichee
5. premium : True si article reserve aux abonnees (icone cadenas)

```
def extraire_articles(driver) -> list[dict]:
    articles = driver.find_elements(By.CSS_SELECTOR,
        "article, [class*='article-item'], [class*='card-article']")
    resultats = []
    for art in articles[:20]:
        try:
            titre = art.find_element(By.CSS_SELECTOR, "h2, h3, [class*='title']").text.strip()
            if not titre: continue
            rubrique = ""
            try:
                rubrique = art.find_element(By.CSS_SELECTOR,
                    "[class*='rubrique'], [class*='section'], [class*='category']").text.strip()
            except: pass
            chapeau = ""
            try:
                chapeau = art.find_element(By.CSS_SELECTOR,
                    "p, [class*='chapo'], [class*='intro']").text.strip()[:300]
            except: pass
            heure = ""
            try:
                heure = art.find_element(By.CSS_SELECTOR, "time, [class*='date']").text.strip()
            except: pass
            # Detecter le contenu premium (icone cadenas)
            premium = bool(art.find_elements(By.CSS_SELECTOR,
                "[class*='premium'], [class*='abonne'], svg[class*='lock']"))
            resultats.append({
                "titre": titre, "rubrique": rubrique,
                "chapeau": chapeau, "heure_publi": heure, "premium": premium,
            })
        except Exception as e:
            pass
    return resultats

arts = extraire_articles(driver)
driver.quit()
with open("lesechos.json", "w", encoding="utf-8") as f:
    json.dump(arts, f, indent=2, ensure_ascii=False)
print(f"{len(arts)} articles exportes")
```

Bonus - Comparaison headless vs normal

```
# Comparer les temps headless vs normal
t0 = time.time()
d2 = make_driver(headless=True)
d2.get("https://www.lesechos.fr")
WebDriverWait(d2, 15).until(
    EC.presence_of_element_located((By.CSS_SELECTOR, "article")))
t_headless = time.time() - t0
d2.quit()

print(f"Normal   : {t_normal:.1f}s")
print(f"Headless: {t_headless:.1f}s")
print(f"Gain     : {t_normal/t_headless:.1f}x plus rapide")
# Attendu : gain ~2-3x en headless
```

CHECKLIST AVANT RENDU

- [] doctolib.json contient ≥ 5 medecins avec les 5 champs
- [] lesechos.json contient ≥ 10 articles avec les 5 champs
- [] WebDriverWait utilise (aucun time.sleep() fixe dans le flux principal)
- [] Screenshot capture en cas d'echec (try/except + save_screenshot)
- [] README.md : pourquoi Selenium et pas requests ? gain headless mesure

Pour aller plus loin -- Defis autonomes

Trois defis qui exigent d'ouvrir un vrai navigateur, d'observer des sites reels et de prendre des decisions en contexte. Impossible a faire 'de tete' ou avec une IA seule.

Defi 1 -- Cookie forensics en conditions reelles

Apres avoir clique 'Accepter tout' sur Doctolib, ouvrez DevTools > Application > Cookies. Analysez les cookies deposes.

- Identifiez 3 cookies de domaine tiers (tracking, analytics). Pour chacun : nom, domaine, duree de vie, valeur encodee ou non.
- Reperez le cookie exact que votre Strategie 2 (injection directe) doit reproduire pour bypasser la banniere sans cliquer. Quel est son nom et quelle valeur lui donner ?
- Comparez avec un autre site de prise de rendez-vous (Maiia, Qare, Livi). Les memes cookies de consentement sont-ils presents ? Meme nom ?

Ce que cette analyse apporte

Comprendre ce que vous acceptez reellement quand vous cliquez 'Tout accepter'.

Identifier pourquoi l'injection de cookie (Strategie 2) est plus robuste que le clic.

Reperer les cookies cross-site qui permettent le suivi inter-domaines (RGPD Art. 5).

Defi 2 -- Empreinte anti-bot de votre driver

Des sites detektent Selenium a la lecture de proprietes JavaScript injectees par le driver. Testez votre propre detectabilite.


```
# Lancez votre driver et naviguez vers un detecteur public
from selenium import webdriver
opts = webdriver.ChromeOptions()
# Tester 1 : sans flag anti-detection
driver = webdriver.Chrome(options=opts)
driver.get("https://bot.sannysoft.com")
driver.save_screenshot("bot_normal.png")
driver.quit()

# Tester 2 : avec flags anti-detection
opts2 = webdriver.ChromeOptions()
opts2.add_argument("--disable-blink-features=AutomationControlled")
opts2.add_experimental_option("excludeSwitches", ["enable-automation"])
driver2 = webdriver.Chrome(options=opts2)
driver2.get("https://bot.sannysoft.com")
driver2.save_screenshot("bot_stealth.png")
driver2.quit()
```

- Comparez les deux screenshots : quels champs passent de rouge a vert ?
- Le champ 'webdriver' est-il toujours detecte en mode stealth ?
- Testez aussi en mode headless : quels nouveaux champs deviennent rouges ?

Defi 3 -- Robustesse face aux changements de site

Revenez sur votre scraper Doctolib 3 jours apres la premiere ecriture et relancez-le sans modification. Observez ce qui se passe.

- Notez exactement quelle exception est levee (ou si tout passe). Quel selecteur CSS a change ou disparu ?
- Implementez un mecanisme de fallback : si le selecteur principal echoue, essayer un selecteur de secours. Exemple :

```
def trouver_nom(carte):
    # Selecteur principal
    el = carte.find_elements(By.CSS_SELECTOR, "h2[class*='name']")
    if el: return el[0].text.strip()
    # Fallback 1
    el = carte.find_elements(By.CSS_SELECTOR, "h3, [data-test*='name']")
    if el: return el[0].text.strip()
    return "n/a" # Fallback final
```

- Appliquez ce pattern a tous les champs fragiles. Combien de selecteurs ont effectivement change en 3 jours ?